Local Dimension Regularization in Local Polynomial Smoothing

From a moving line fit, to charts on a manifold, to the stratified geometry of omics data — and why the local dimension wants to be regularized with ℓ_1, not ℓ_2

Internal expository note — LPS / `geosmooth` program

source: `~/current_projects/geosmooth/dev/notes/foundations/lps_local_dimension_regularization.tex`

2026-06-15 12:43:02 ET

Abstract

This note follows a single thread from beginning to end. We start with the most elementary smoother imaginable — fit a straight line in a small moving window — and watch it grow, step by step, into the local polynomial smoother (LPS) used in `geosmooth`: a method that lays a little coordinate chart over each point of a data cloud and fits a local polynomial on it. The pivotal quantity in that construction is the *local dimension* of the chart, and the pivotal difficulty is that the local dimension has to be *estimated*, anchor by anchor, from a handful of nearby points — so the estimates are noisy. The natural cure is to let neighboring anchors share information. But the moment we try, the geometry of real data — especially compositional omics data, which lives on the boundary of a simplex and is genuinely *stratified*, with the dimension jumping from place to place — tells us *how* to share: not by smoothing (which blurs the jumps), but by a sparsity-of-changes penalty that keeps the jumps sharp. That single observation, that the dimension field should be regularized with ℓ_1 rather than ℓ_2, opens onto a surprisingly deep and well-developed body of theory, from manifold regression to stratification learning to exact combinatorial optimization. The note is written to be read in order, with the mathematics stated precisely and every idea also explained in plain words and illustrated by a figure. Each load-bearing reference gets a short appendix that explains its core idea from scratch.

A note on the figures. Every figure in this document is an original illustration generated specifically for it (the generating script lives beside the source). None is reproduced from any paper; they are simple, deliberately schematic pictures meant to make one idea visible at a time.

Contents

1 The question, and a smoother that answers it

The problem underneath everything in this note is the oldest one in statistics dressed in modern clothes: given a feature vector X and a response Y, estimate the *conditional expectation*

$$f(x) \;=\; \mathbb{E}[\,Y \mid X = x\,].$$

When Y is continuous, f is a regression surface; when Y is binary, $f(x) = \Pr(Y = 1 \mid X = x)$ is a probability surface. In the applications that motivate `geosmooth` — 16S rRNA microbiome profiles, metagenomes, other omics snapshots — X is high-dimensional and lives on a geometrically complicated set: not a nice filled-in region of space, but a thin, curved, often non-manifold object. We want a single estimator that recovers f well on that kind of set, for both binary and continuous features, without assuming we know its shape in advance.

The local polynomial smoother answers this question with an idea that is almost embarrassingly simple, and the simplicity is the point. To predict f at a point x, look only at the training points near x, fit a low-degree polynomial to them (weighting the nearer ones more heavily), and read off

the value of that polynomial at x. Do this at every point and you get a smooth surface. Figure 1 shows the whole pipeline at a glance; the rest of this section unpacks it.

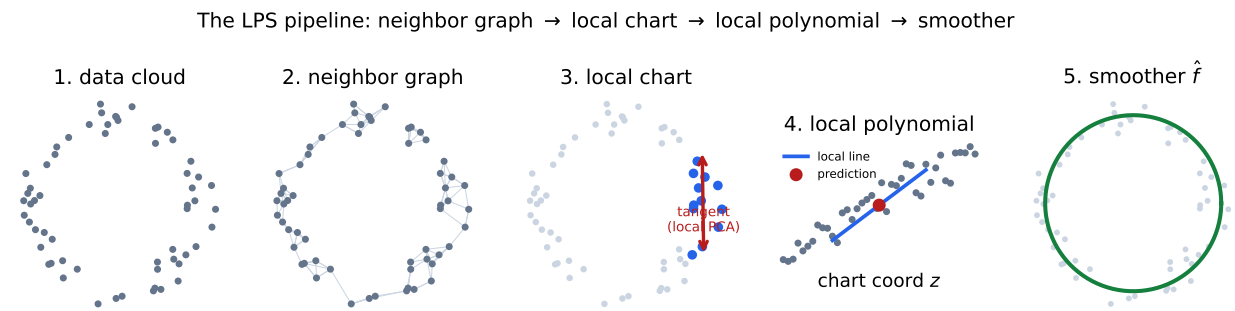

Figure 1: The LPS pipeline. (1) A cloud of data points. (2) A neighbor graph records, for each point, who its nearest neighbors are. (3) At an anchor point, the nearby points define a local chart — a small coordinate system, here a tangent direction found by local PCA. (4) On that chart we fit a weighted local polynomial and take its value at the anchor as the prediction. (5) Repeating at every point yields the smoother $\hat{f}$.

In plain terms Think of estimating the height of a hilly landscape you can only sample at scattered points. To guess the height at a new spot, you do not fit one global formula to the whole range of mountains; you look at the handful of measurements nearest that spot and fit a little tilted plane to them. The smoother is just "do that everywhere."

2 The local polynomial smoother, precisely

Fix a point x_0 at which we want to predict. The smoother has four ingredients.

1. A neighborhood. Choose the k training points nearest x_0 (in some metric). The number k is the *support size*; it plays the role of a bandwidth and controls how local the fit is.

2. Kernel weights. Weight each neighbor x_j by a kernel $w_j = K\big(\|x_j - x_0\|/h\big)$ that decays with distance, where the bandwidth h is typically the distance to the farthest neighbor in the support. Nearer points count more.

3. A local polynomial. In local coordinates $z = z(x)$ centered at x_0 (so $z(x_0) = 0$), build the polynomial design $\phi(z) = (1, z_1, \dots, z_d, z_1^2, \dots)^\top$ up to a chosen degree p, and solve the weighted least squares problem

$$\widehat{\beta} \;=\; \arg\min_{\beta} \sum_j w_j \left(y_j - \phi(z_j)^\top \beta\right)^2 \;=\; (X^\top W X)^{-1} X^\top W y, \tag{1}$$

where X stacks the rows $\phi(z_j)^\top$ and $W = \mathrm{diag}(w_j)$.

4. The prediction. Because the chart is centered at x_0, the polynomial's value there is simply its intercept:

$$\hat{f}(x_0) \;=\; \phi(0)^\top \widehat{\beta} \;=\; e_1^\top \widehat{\beta} \;=\; \widehat{\beta}_0. \tag{2}$$

The other coefficients (the local slope and curvature) are nuisance parameters; they exist only to make the intercept unbiased. The smoother is *linear* in the responses, $\hat{f}(x_0) = \sum_j S_j(x_0)\, y_j$, a fact that quietly powers much of the theory.

Degree $p = 0$ is just a weighted local average (the Nadaraya–Watson estimator). Degree $p = 1$ fits a local line or plane, and that one extra degree buys a famous advantage at the edges of the data, illustrated in Figure 2 and Figure 3: a local average lags behind a trend near a boundary (it can only average points that lie to one side), whereas a local line extrapolates the trend correctly. This is the first hint that the *model* fitted locally, not just the bandwidth, matters.

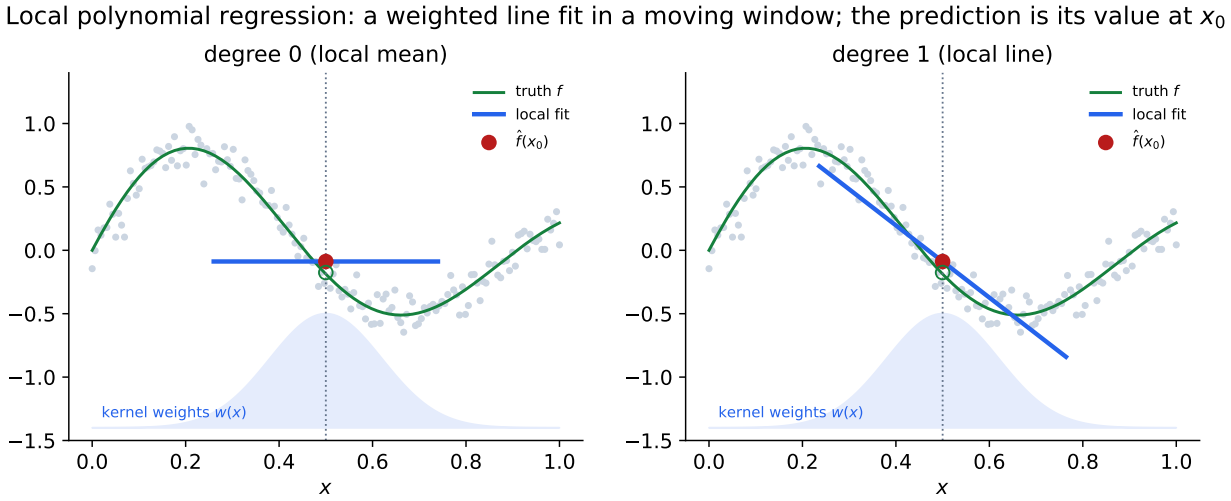

Figure 2: Local polynomial regression at a single query point x_0 (dotted line). The blue shaded curve is the kernel weight; nearer points count more. Left: degree 0 fits a local mean. Right: degree 1 fits a local line and reads off its value at x_0. The open green circle is the truth, the red dot the prediction.

The smoother has a few tuning choices — support size k, degree p, kernel shape — and LPS selects them by cross-validation, refitting on held-out folds and keeping the combination with the smallest held-out error. Everything so far is classical: this is local polynomial regression as developed over decades (Stone; Cleveland's LOWESS; Fan and Gijbels; Loader's `locfit`). What makes the `geosmooth` version interesting is *where* the local coordinates z come from when the data live on a complicated set — the subject of the next section.

In plain terms Degree 0 says "the surface is locally flat," degree 1 says "locally tilted," degree 2 says "locally curved." Allowing a tilt is what saves you at the edge of the data: an average of points that are all on your left will underestimate a surface that keeps rising to your right, but a line through them will not.

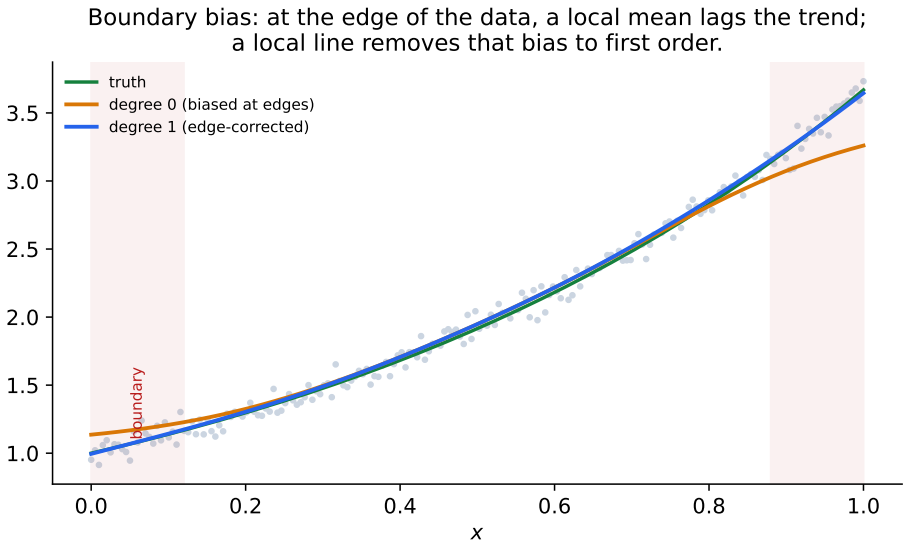

Figure 3: The boundary-bias advantage of going from degree 0 to degree 1. Near the edges of the data (red bands) a local mean systematically lags the rising trend, because all the points it can average lie to one side; a local line extrapolates the trend and stays unbiased to first order. This is why LPS prefers local linear and quadratic fits.

3 Living on a manifold: charts and intrinsic dimension

If the data filled an ordinary region of $\mathbb{R}^D$, the local coordinates z could just be the ambient coordinates. But high-dimensional omics data do not fill $\mathbb{R}^D$. A microbiome sample with D taxa is a point in a D-dimensional space, yet the samples that actually occur cluster near a thin, curved subset of much lower dimension — a *manifold*, at least locally. Estimating f as if the data were D-dimensional wastes everything: there are never enough neighbors to fill a high-dimensional ball, and the fit drowns in empty directions.

The fix is to build the local chart out of the data themselves. Take the k neighbors of an anchor, center them, and run *principal component analysis*: the top few principal directions span the local tangent plane of the underlying set, and projecting the neighbors onto them gives low-dimensional chart coordinates $z \in \mathbb{R}^d$ with $d \ll D$. Figure 4 shows the picture: a curved 2-D surface sitting in 3-D, and the flat tangent chart that local PCA recovers near a point. The local polynomial of Section 2 is then fitted on that d-dimensional chart instead of in the ambient D-dimensional space.

The number d — how many principal directions to keep — is the *local intrinsic dimension*, and it is read from the eigenvalues of the local covariance. If the neighborhood truly lies near a d-dimensional plane, the covariance has d large eigenvalues (variation along the surface) and the rest near zero (thickness, i.e. noise). A clear *gap* in the eigenvalue spectrum names the dimension; Figure 5 and Figure 6 make this concrete.

In plain terms A pile of beads strung on a wire is technically a cloud of points in 3-D space, but it is really one-dimensional: it only varies *along* the wire. PCA measures variation in each direction; if almost all the variation is along one direction and almost none across, the cloud is one-dimensional. The "intrinsic dimension" is the number of directions in which the data genuinely move.

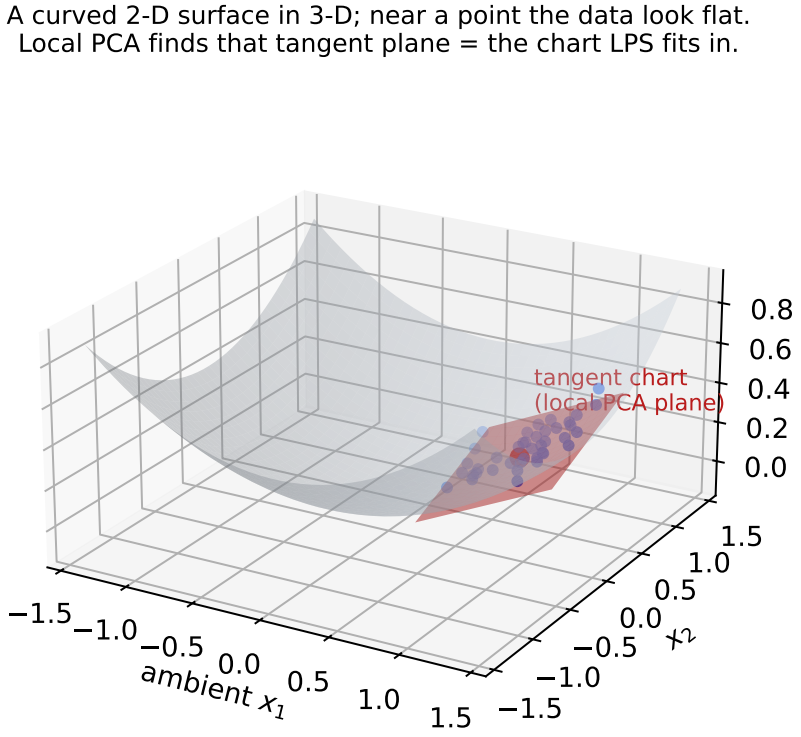

Figure 4: Why charts. A curved two-dimensional surface embedded in three dimensions. Near any point the data look flat; local PCA finds that flat tangent plane (red), and LPS fits its local polynomial in those two chart coordinates rather than in the three ambient ones.

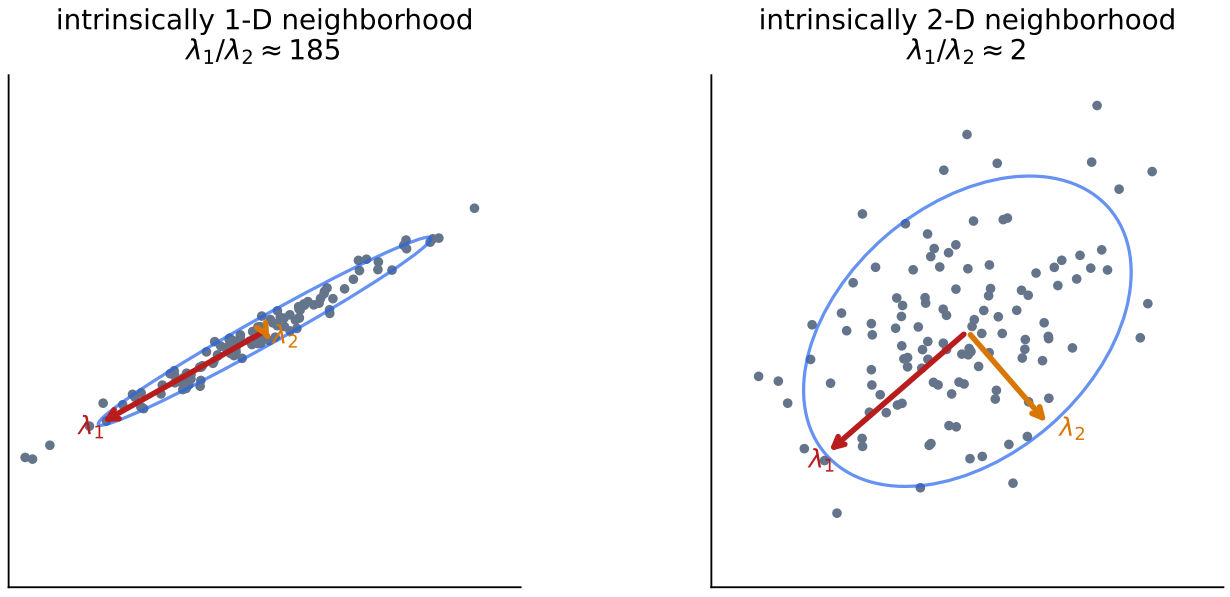

Figure 5: Local PCA. Left: a neighborhood that is essentially one-dimensional — the spread collapses onto a single direction, so $\lambda_1 \gg \lambda_2$. Right: a genuinely two-dimensional neighborhood, with two comparable eigenvalues. The ratio λ_1/λ_2 is the signal that distinguishes the two cases.

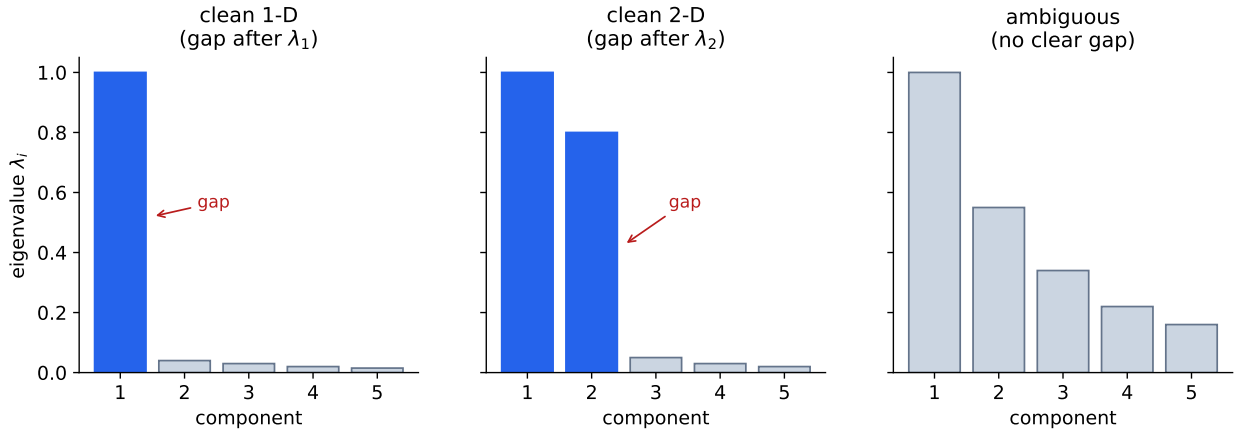

Figure 6: The eigenvalue spectrum ("scree") is how the local dimension is read off. A large gap after the k-th eigenvalue says the intrinsic dimension is k. The rightmost panel is the trouble to come: from a small, noisy neighborhood the gap is often not clear at all.

Historical context Fitting a local polynomial in an estimated tangent chart is not new; it is the established method of *manifold local polynomial regression*. Bickel and Li (2007) proved that local polynomial regression automatically adapts to a low-dimensional manifold: its error rate is governed by the intrinsic dimension d, not the ambient dimension D (Appendix A). Cheng and Wu (2013) made the construction explicit — reduce to the intrinsic dimension by local PCA, then fit a local linear model on the tangent plane — and worked out its bias and variance, including the subtle extra bias that the manifold's curvature injects (Appendix B). LPS with `coordinate.method = "local.pca"` is a careful, engineered instance of exactly this idea.

4 Why the local dimension is the whole game

It is tempting to treat the chart dimension as an implementation detail. It is not; it is the single most consequential number in the method. The reason is the convergence rate. For estimating a smooth (p-times differentiable) regression function on a d-dimensional set, the best possible root-mean-square error scales like

$$\text{RMSE} \asymp n^{-p/(2p+d)}, \tag{3}$$

which for local-linear fits ($p = 2$ effective smoothness) is $n^{-2/(d+4)}$. The *intrinsic* d sits in the exponent. Figure 7 plots these rates: in dimension 1 the error falls steeply with sample size; by dimension 3 it crawls. Getting d right is therefore worth far more than any amount of tuning elsewhere, and *over*-estimating d — keeping an empty, noisy direction as if it carried signal — directly slows the achievable rate.

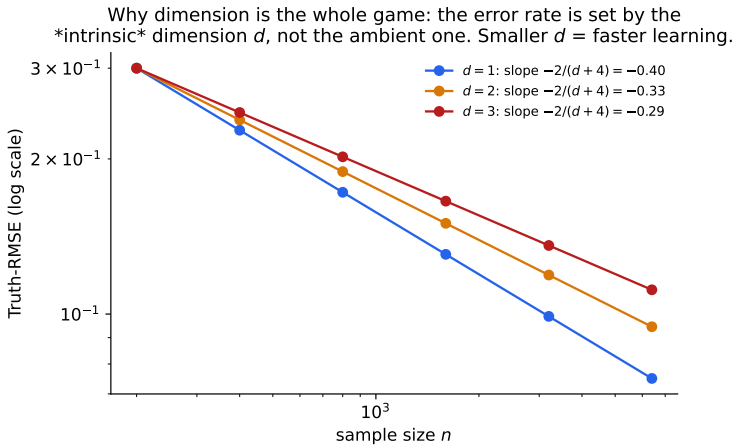

Figure 7: The error rate $n^{-2/(d+4)}$ is set by the intrinsic dimension d. Lower-dimensional structure is learned dramatically faster. This is why estimating d correctly — and not inflating it — is the highest-leverage decision in the whole pipeline.

Historical context Kpotufe (2011) sharpened this in exactly the direction this note cares about: he showed that k-nearest-neighbor regression adapts not merely to a global intrinsic dimension but to the *local* intrinsic dimension, with an error at a query point that depends only on how mass accumulates in balls around *that* point (Appendix C). In other words, the right notion of dimension is local and can vary across the space — precisely the regime LPS's `chart.dim = "local.auto"` mode is built for, and precisely why getting the *local* estimate right is the crux of the next section.

5 The trouble: anchor estimates are noisy

Here the story turns. The local dimension is the most important number in the method, and it must be estimated separately at every anchor from the k points in that anchor's neighborhood — typically k between 10 and 30. Estimating a dimension from a dozen points is a hard statistical problem, and every estimator of it inherits high variance from the small sample.

The simplest estimator is the eigenvalue gap of Section 3: count the principal directions before the spectrum falls off a cliff. A more principled one is the Levina–Bickel maximum-likelihood estimator, which reads the dimension off the *growth rate of nearest-neighbor distances* — in a d-dimensional set the number of points within radius r grows like r^d, so d can be recovered by maximum likelihood from the observed neighbor distances (Appendix D). Whatever the estimator, the difficulty is the same: from a small neighborhood the eigenvalue gap is mushy and the distance growth rate is noisy, so the per-anchor estimate $\widehat{d}_i$ scatters.

Figure 8 shows the symptom on a deliberately simple test object: a flat two-dimensional sheet with a one-dimensional filament attached (true dimensions on the left). With a small neighborhood ($k = 8$) the estimated dimension map is a speckle of errors — the sheet is peppered with points mistakenly called one-dimensional, the filament with points called two- or three-dimensional. Enlarging the neighborhood ($k = 35$) calms the speckle but starts to smear the boundary between the strata, because a big neighborhood straddles both. Figure 9 makes the trade-off explicit: the misclassification rate is high for small k (variance) and rises again for large k (the neighborhood crosses structure, i.e. bias). There is no single good k.

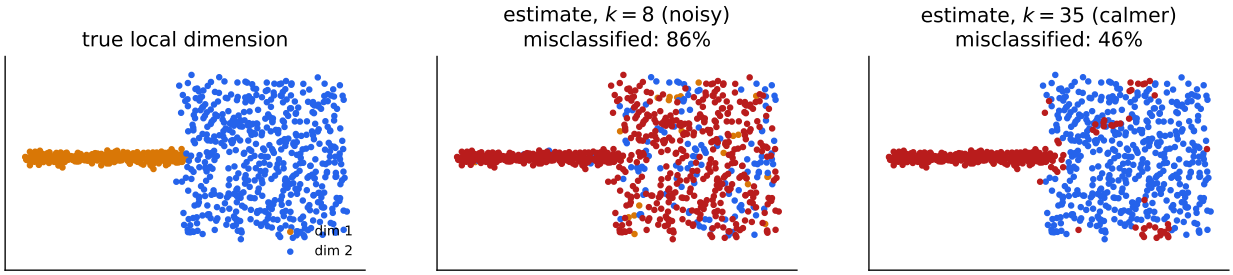

Figure 8: Per-anchor dimension estimates are noisy. A two-dimensional sheet with a one-dimensional filament attached. Independent per-anchor estimation with a small neighborhood ($k = 8$, middle) speckles the map with errors; a larger neighborhood ($k = 35$, right) is calmer but blurs the boundary between the strata. Colors: orange = dimension 1, blue = dimension 2, red = 3.

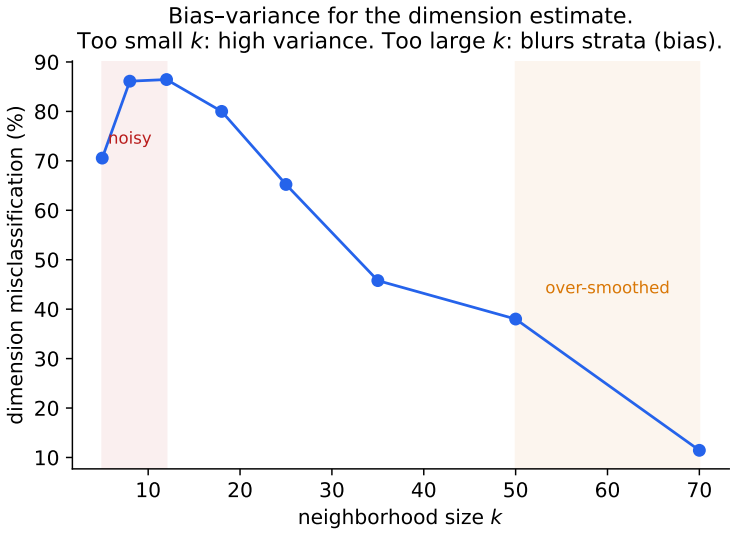

Figure 9: The bias–variance trade-off for the dimension estimate itself. Small neighborhoods give high-variance estimates (speckle); large ones bias the estimate by straddling strata of different dimension. No single neighborhood size is right — which is what forces us to *regularize* rather than just retune.

This is not a hypothetical worry. In `geosmooth`'s own experiments, letting the local dimension float freely per anchor (the `local.auto` mode) sometimes *hurt* accuracy relative to a fixed global dimension, and the damage was traced to anchors that over-estimated their dimension — exactly the empty-direction inflation that Section 4 warned slows the rate. The estimates are not just noisy; their noise has teeth.

In plain terms Asking "what dimension is this neighborhood?" from ten or twenty points is like asking "is this coin fair?" from ten flips: the honest answer is a wide range. Do it independently at thousands of points and you get a noisy, freckled map — some freckles harmless, but some (the over-estimates) actively degrade the prediction. We cannot fix this by changing the neighborhood size alone, because small neighborhoods are noisy and large ones blur real boundaries.

6 The natural idea: let neighbors share information

If a single anchor cannot pin down its dimension from its own dozen points, the obvious move — and the one this whole note is organized around — is to stop treating the anchors as independent. Nearby anchors look at overlapping neighborhoods and almost always have the *same* true local dimension, so they should be allowed to pool their evidence. We already have the object that says who is near whom: the neighbor graph of Figure 1, the same graph the smoother is built on. The proposal is to estimate the dimension not one anchor at a time, but as a single *field* $\{d_i\}$ over that graph, balancing two terms:

$$\{\widehat{d_i}\} \;=\; \arg\min_{\{d_i\}} \; \underbrace{\sum_i c_i(d_i)}_{\substack{\text{local evidence}\\ \text{(fit each anchor's data)}}} \;+\; \lambda \underbrace{\sum_{(i,j)\in E} w_{ij}\,\rho(d_i, d_j)}_{\substack{\text{agreement}\\ \text{(neighbors should match)}}} . \tag{4}$$

The first term is the per-anchor cost from Section 5 — how badly dimension d fits anchor i's own neighborhood (small if the eigenvalue gap or the Levina–Bickel likelihood favors d). The second term, summed over graph edges E, penalizes neighbors for disagreeing; λ sets how hard we lean on agreement. This is a textbook *denoising* setup — fidelity to noisy data plus a smoothness prior on a graph — and it will clearly calm the speckle.

The only real question is the shape of the penalty $\rho(d_i, d_j)$. And that question, innocuous as it looks, is where the geometry of the data forces our hand.

In plain terms Instead of each anchor guessing its dimension alone, we let the whole map vote: keep each anchor close to what its own data suggest, but also close to its neighbors. The tug-of-war between "trust my data" and "agree with my neighbors" is tuned by one knob, λ. The only thing left to decide is how we measure "disagreement" between neighbors — and that turns out to matter enormously.

7 But the geometry is stratified

If the true dimension were the same everywhere, any reasonable agreement penalty would do. It is not. The objects that omics data live near are typically *not manifolds*: they are *stratified spaces*, built by gluing together pieces of different dimension. A cone is a two-dimensional sheet that

pinches to a zero-dimensional point at its tip. The boundary of a high-dimensional simplex — where compositional data live — is a scaffold of faces of every dimension from 0 (vertices) up. Figure 10 shows the canonical toy case: a one-dimensional edge meeting a two-dimensional sheet along a singular junction.

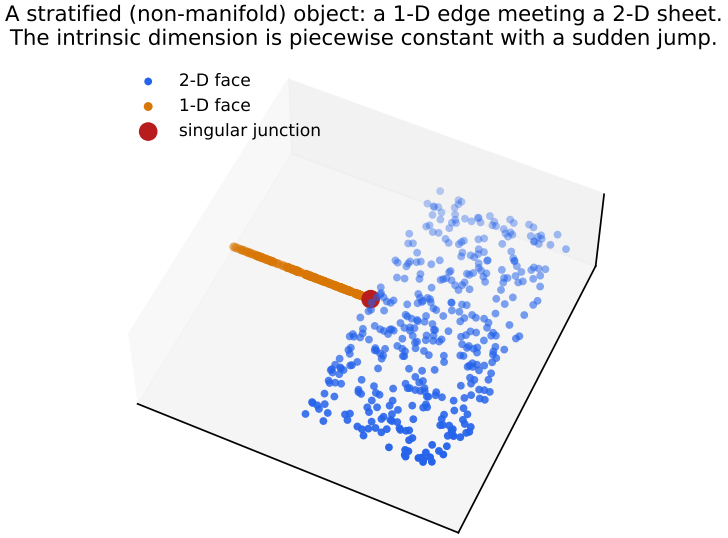

Figure 10: A stratified, non-manifold object: a one-dimensional edge glued to a two-dimensional sheet. The true local dimension is 1 on the edge and 2 on the sheet, with a sudden *jump* at the junction (red). The dimension field is piecewise constant with sharp boundaries, not smoothly varying.

The consequence for our field $\{d_i\}$ is decisive. The true dimension is *piecewise constant*: constant within each stratum, with abrupt jumps where strata meet. It is emphatically *not* smooth. A penalty that wants the field to vary gradually would be fighting the truth, smearing a clean $1 \to 2$ jump into a meaningless gradient of fractional dimensions across the boundary. We do not want to smooth the dimension field; we want to *denoise it while keeping its jumps*. Those are different goals, and they call for different penalties.

In plain terms Real data clouds are often glued together from pieces of different dimension — a one-dimensional thread here, a two-dimensional sheet there, meeting at a seam. The dimension is constant within a piece and then jumps at the seam. So when we clean up the noisy map, we must not blur it like a smudged photograph; we must clean it like a paint-by-numbers, keeping the borders between regions crisp.

8 ℓ_1, not ℓ_2

This is the heart of the note. Two penalties present themselves for the agreement term $\rho(d_i, d_j)$ in (4):

$$\text{squared / Laplacian } (\ell_2): \quad \rho(d_i, d_j) = (d_i - d_j)^2, \tag{5}$$

$$\text{total variation } (\ell_1): \quad \rho(d_i, d_j) = |d_i - d_j|. \tag{6}$$

They look almost the same. They behave oppositely at a boundary, and the difference is exactly the difference between smoothing and denoising-with-edges.

The ℓ_2 penalty (5) is the classical graph-Laplacian smoother. Its minimizer is *smooth*: squared differences are cheap when small and ruinously expensive when large, so the solution avoids any single big jump by spreading the change over many small ones. At a true $1 \to 2$ stratum boundary it therefore produces a gradual ramp — fractional "dimensions" $1.3, 1.6, \ldots$ smeared across a band of anchors — which is both wrong (dimension is an integer) and destructive (it blurs the boundary we most want to find).

The ℓ_1 penalty (6) — total variation, the graph version of the *fused lasso* — behaves entirely differently. The absolute value does not punish a large jump disproportionately, so the solution is free to put all its "change" into a few sharp jumps and stay perfectly flat in between. Its minimizer is *piecewise constant*: it denoises within each stratum and keeps the boundary crisp. This is the same reason total-variation denoising preserves edges in images while Gaussian blur destroys them.

Figure 11 is the whole argument in one picture. Along a path crossing a stratum boundary, the true dimension is a clean step from 1 to 2; the per-anchor estimates are a noisy cloud; the ℓ_2 fit is a smooth ramp that blurs the boundary; the ℓ_1 fit is a clean step that lands almost exactly on the truth.

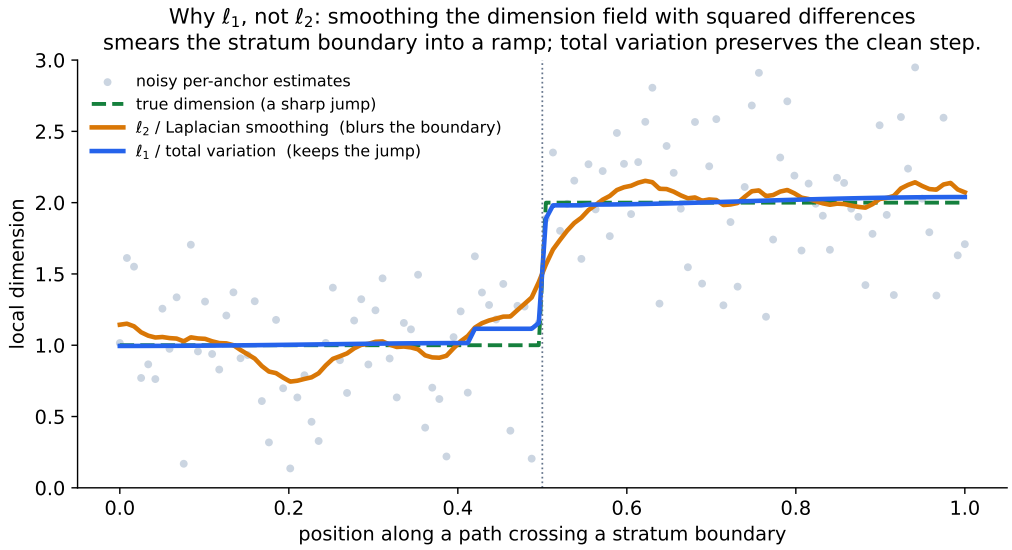

Figure 11: The central figure. Cleaning the noisy dimension field with squared differences (ℓ_2, orange) smears the stratum boundary into a ramp; with total variation (ℓ_1, blue) it stays a clean step that tracks the true jump (green dashed). Match the penalty to the nature of the field: the dimension field is piecewise constant, so use ℓ_1.

The same effect on a two-dimensional graph is shown in Figure 12: independent per-anchor estimates are a speckled mess, and a total-variation field on the neighbor graph recovers clean regions separated by a sharp, wavy boundary.

In plain terms Squared penalties hate big jumps, so to avoid one they dribble the change out over a wide blurry band — fine for a quantity that really does vary smoothly, fatal for one that is supposed to jump. Absolute-value penalties are indifferent between one big jump and many small ones, so they happily keep a single clean edge and stay flat elsewhere. Since a dimension map is paint-by-numbers (flat regions, sharp borders), the absolute-value (ℓ_1) penalty is the right tool, and the squared (ℓ_2) one actively erases the borders we care about.

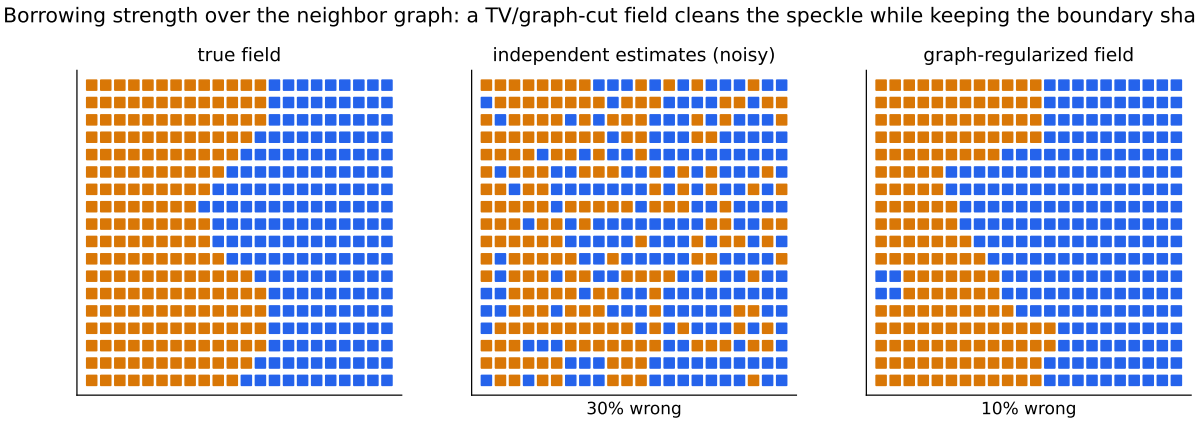

Figure 12: Borrowing strength over the neighbor graph in two dimensions. Left: the true dimension field, two regions split by a wavy boundary. Middle: independent per-anchor estimates — a quarter of the anchors are wrong. Right: the total-variation / graph-cut field cleans the speckle almost perfectly while keeping the boundary sharp.

Historical context The ℓ_1-on-differences idea is the *fused lasso* of Tibshirani, Saunders, Rosset, Zhu and Knight (2005): penalize the absolute differences of neighboring coefficients to obtain piecewise-constant estimates with a sparse set of jumps (Appendix E). On a general graph it becomes total-variation denoising / the graph fused lasso. It is the same mathematics that underlies edge-preserving image denoising, and `geosmooth` already carries graph trend-filtering machinery of this family — so the dimension-field idea reuses tooling the program owns.

9 There is no single "true" dimension: scale

Before we can solve (4) we must confront a subtlety that the clean picture above hides: the local dimension is not even well defined until we fix a *scale*. Real strata are not infinitely thin. A "one-dimensional" filament in a microbiome embedding is really a very thin tube; a "two-dimensional" sheet has a slight thickness of noise. Whether such a tube reads as one- or two-dimensional depends on how closely you look, as Figure 13 shows. At a coarse scale (a large bandwidth, larger than the tube's radius ρ) the thickness is invisible and the tube is one-dimensional; at a fine scale (smaller than ρ) you resolve the thickness and it is two-dimensional. The "correct" dimension is a function of scale, not a single number.

This sounds like a problem but it resolves cleanly, and the resolution is a principle worth stating plainly: *tie the dimension scale to the smoother's bandwidth.* The chart only has to be faithful at the scale the local polynomial actually uses; so estimate the per-anchor cost $c_i(d)$ from the eigenvalue profile measured at the bandwidth radius, not at some arbitrary scale. A refinement is to look across a band of scales and trust a dimension that is *persistent* — stable across scales is signal, a dimension that only collapses at large scale is a coarse-graining artifact. This persistence idea is the local, bandwidth-anchored cousin of multi-scale intrinsic-dimension and singularity detection (Appendix K).

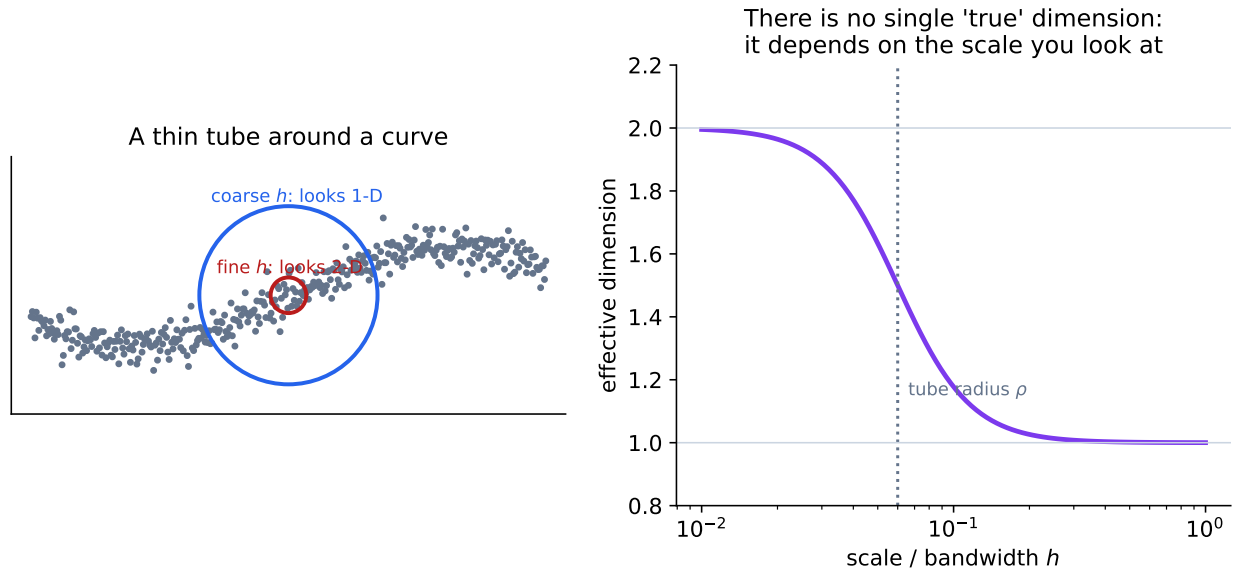

Figure 13: Scale-dependence. Left: a thin tube around a curve. Viewed through a coarse window (blue, wider than the tube radius ρ) it looks one-dimensional; through a fine window (red) it looks two-dimensional. Right: the effective dimension as a function of the viewing scale is a smooth descent from 2 to 1 as the window grows past ρ — there is no scale-free answer.

In plain terms Zoom out on a thread and it is a line; zoom in and you see it has thickness. Asking "what dimension is it?" has no answer until you say how closely you are looking. The clean fix: look at exactly the scale your smoother smooths at, and only believe a dimension that survives across a range of zoom levels.

10 Solving the field exactly: dimension is an ordered integer

We now have a clean optimization problem, (4) with the total-variation penalty (6). It would be a shame to solve it with a heuristic, and we do not have to, because of a structural gift: the dimension labels live in a small, *linearly ordered* integer set $\{1, 2, \ldots, D_{\max}\}$. That ordering unlocks exact combinatorial optimization.

The relevant fact is Ishikawa's theorem: a first-order Markov random field whose pairwise cost is a *convex* function of the difference of *ordered* labels can be minimized *exactly* and in polynomial time by computing a single minimum cut on an auxiliary layered graph. The total-variation cost $|d_i - d_j|$ is convex, so our field is in scope. The construction stacks a column of nodes for each anchor, one per candidate dimension, wires them with edges whose capacities encode c_i and $\lambda|d_i - d_j|$, and reads the optimal dimension at each anchor off where the minimum cut slices its column (Figure 14). No continuous relaxation, no rounding, no local minima — the global optimum of an integer-valued, edge-preserving dimension field, from one graph cut.

Two variations are worth naming. If instead of total variation one prefers the *Potts* penalty $\rho(d_i, d_j) = \mathbf{1}\{d_i \neq d_j\}$ — "any jump, large or small, costs the same," the purest "partition into constant regions" prior — then exact optimization becomes NP-hard for more than two labels, but the α-expansion graph-cut algorithm of Boykov, Veksler and Zabih delivers a solution within a factor of two of optimal, fast and in practice excellent (Appendix G). And if one wants to respect

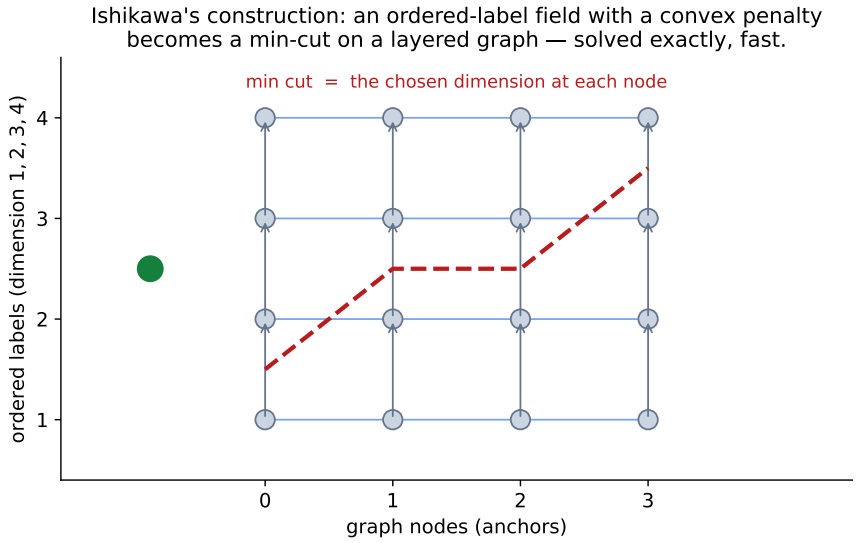

Figure 14: Ishikawa's construction. Each anchor becomes a column of nodes, one per candidate dimension (an ordered label set). A convex penalty on dimension differences makes the energy a min-cut problem on this layered graph; the cut (red dashed staircase) selects one dimension per anchor and is the exact global optimum.

the directional structure of strata (next section), there is machinery for convex priors on *partially* ordered labels (Appendix H).

In plain terms Because a dimension is a whole number from a short list, choosing the best whole number at every point of the graph — balancing each point's own evidence against agreement with neighbors — is not a hopeless combinatorial search. There is an exact, fast algorithm (a "minimum cut") that finds the single best map in one shot. You do not have to guess-and-round a continuous approximation.

Historical context Ishikawa (2003) proved the exact graph-cut result for convex priors on ordered labels (Appendix F); Boykov, Veksler and Zabih (2001) gave the α-expansion approximation for the harder Potts-type energies (Appendix G); Domokos, Schmidt and Cremers (2018) extended exact convex-prior optimization to partially ordered label sets (Appendix H). These come from computer vision, where labeling a grid of pixels by depth or segment is the same mathematics as labeling a graph of anchors by dimension.

11 The deeper structure: strata have a partial order

Total variation treats a $1 \to 2$ jump and a $2 \to 1$ jump identically, but real stratifications are not symmetric. A proper (Whitney) stratification obeys a *frontier condition*: the boundary of a k-dimensional stratum is made of strata of dimension strictly less than k. The closure of a two-dimensional face contains its bounding edges and vertices, never the other way around. So dimension is *lower-semicontinuous*: as you approach a seam, the dimension can only drop toward the seam, and the lower-dimensional stratum sits inside the closure of the higher one. A symmetric penalty is blind to this; it would happily accept a lone island of "dimension 2" surrounded by "dimension 1," which is geometrically impossible for a face lattice.

The faithful version of the dimension field therefore encodes the strata as a *partial order* (the face lattice of the simplex is literally a poset), and penalizes transitions in a way that respects it — which is exactly the setting of convex priors on partially ordered labels (Appendix H). One does not have to go this far to get most of the benefit, but it is the principled endpoint, and it connects the whole construction to a recognized research area.

That area is *stratification learning*: recovering, from a noisy point cloud, the pieces of different dimension that a singular space is glued from. Haro, Randall and Sapiro framed it as simultaneously estimating a soft clustering, a local intrinsic dimension and a density, going beyond single-manifold learning (Appendix I). A topological-data-analysis branch uses *local homology* — the homology of a small neighborhood with its center removed, which fingerprints the local structure of a singularity — to detect strata and their dimensions (Bendich, Wang and Mukherjee; Appendix J), with multi-scale descendants that detect singularities by how local geometry changes across scales (Appendix K). These supply, and can sanity-check, the per-anchor cost $c_i(d)$ that feeds our field.

In plain terms Strata fit together with a built-in pecking order, like the parts of a solid shape: edges bound faces, faces bound the interior, never the reverse. So a faithful cleanup should know that a dimension can drop as you reach a seam but a stray high-dimensional speck stranded in a low-dimensional region is nonsense. Detecting these glued-together pieces from data is a studied problem in its own right, called stratification learning.

12 The compositional twist: omics data on the simplex

Everything so far applies to any data. Compositional omics data add a twist that is at once a gift and a trap. A microbiome sample is a vector of relative abundances summing to one — a point in a simplex — and most taxa are genuinely *absent* from most samples, not merely rare. Those structural zeros place the data exactly on the lower faces of the simplex, where some coordinates vanish. Which face a sample lies on is told by its *support*: the set of taxa with nonzero abundance (Figure 15, left).

The gift is that the support gives the dimension *for free as an upper bound*: a sample with m nonzero taxa lies on an $(m-1)$-dimensional face, so its local dimension is at most $m-1$. No estimation needed for the cap. The trap is that the cap is only an upper bound. As Figure 15 (right) shows, the data can have full support yet concentrate on a thin tube near a lower face — so the *effective* dimension is smaller than the face's nominal dimension, and *that* must still be estimated. The structural zeros tell you the room the data could fill; they do not tell you how much of it the data actually fill.

There is also a genuine trap hidden in the coordinates. The natural compositional coordinates are log-ratios, $u_i = \log(x_i/x_{\text{ref}})$, and as a taxon's abundance heads toward zero (toward a face), $u_i \to -\infty$ with *growing* variance. On the log scale a taxon approaching the boundary looks like the *largest* source of variation — the exact opposite of the thin, collapsing direction it geometrically is (Figure 16). Estimating the local dimension by PCA on log-ratios would therefore *inflate* the dimension near faces, hiding the very concentration we want to find. The cure is to estimate the dimension on a coordinate where approaching the boundary reads as a variance collapse — the raw or cube-scaled abundance — even while modeling fine dependence among the present taxa on the log scale.

This is also where the present note meets the companion *chart-aware local association* design, whose "face stratification" is exactly the combinatorial (support-based) stratification described here. The local-dimension field is the statistical refinement of that combinatorial scaffold: it softens

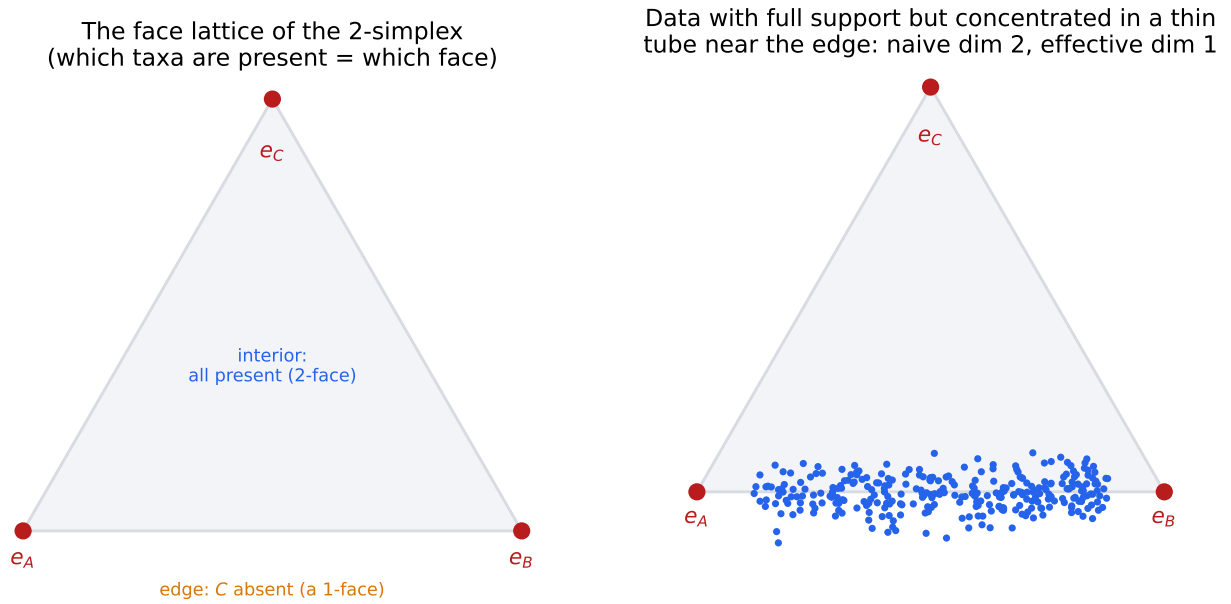

Figure 15: The compositional twist. Left: the face lattice of the simplex — which taxa are present tells you which face a sample lies on, and a sample on a 1-dimensional edge has dimension at most 1. Right: but a sample can have *full* support (all taxa present) and still concentrate in a thin tube near a lower face, so its effective dimension is below the face's nominal dimension.

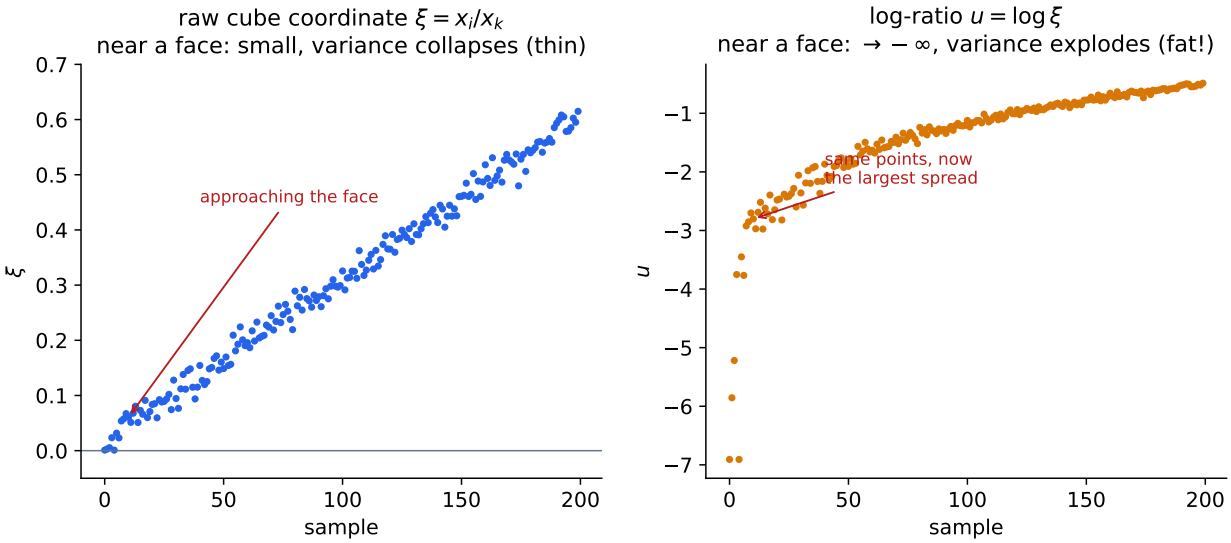

Figure 16: The coordinate trap. As a taxon's abundance approaches zero (approaching a face), the raw coordinate ξ collapses toward 0 with shrinking variance (left, "thin"), but the log-ratio $u = \log \xi$ runs to $-\infty$ with exploding variance (right, "fat"). The same points look low-variance on one scale and dominant on the other. Estimate the dimension on the raw scale.

the hard presence/absence threshold into a scale-aware effective dimension, and it supplies the within-face effective rank that the association model's covariance layer should actually use.

In plain terms In microbiome data, "which microbes are present" is known exactly and already tells you the most the dimension could be. But present is not the same as active: a handful of present microbes can still trace out a thin curve rather than fill a region, so the real dimension can be lower and must be measured. And beware the log scale beloved of compositional analysis: it makes an almost-absent microbe look like the biggest player in the room, which is precisely backwards for judging dimension.

13 Honest limits

Two cautions keep the program honest. First, **detectability has a floor**. Whether a thin concentration is real structure or just measurement noise cannot always be decided from a small neighborhood. In 16S data the "thickness" of a near-face tube may simply be finite-read-depth multinomial noise; from ten or twenty local points one cannot cleanly separate a genuinely k-dimensional concentration from a $(k{+}1)$-dimensional process observed with thin-looking noise. The principled escapes are the multi-scale persistence of Section 9 and *replicates* — technical or biological — which estimate the noise floor and let one ask whether a thin direction survives above it. Absent those, "the dimension here is k" is meaningful only with an attached scale and noise model.

Second, **the regularization strength λ has no purely internal criterion**. The dimension field is a *means*, not an end: its job is to make the downstream smoother more accurate, so λ (and the dimension scale) should be chosen by held-out predictive error of the LPS that uses the field, not by any intrinsic "dimension-fit" score. This is the same lesson the `geosmooth` program already learned about total local cross-validation scores: a quantity that looks like it measures local quality is not automatically a good global selector.

In plain terms Sometimes a thread really is just a fuzzy point, and no amount of cleverness can tell the difference from a few measurements — you need repeats, or to look across scales. And since the dimension map exists only to help the final prediction, the right way to set its one knob is to ask which setting makes the prediction best, not which makes the map prettiest.

14 Synthesis: one program, two kinds of regularity

Step back and the whole `geosmooth` program comes into focus as a study of two different *kinds* of regularity, each wanting its own penalty.

The *prediction* surface — the conditional mean f, and in the prediction-synchronized smoother (PS-LPS) the agreement of overlapping charts' predictions — is a continuous object that varies smoothly. The right coupling for it is ℓ_2: PS-LPS ties neighboring charts' predictions together with a squared-overlap penalty, and that is correct, because a regression surface should not jump.

The *dimension / support* structure — which stratum you are on, how many directions the data fill, which taxa are present — is a discrete object that is piecewise constant and reorganizes *sharply* at boundaries. The right coupling for it is ℓ_1: total variation on the neighbor graph, solved exactly by graph cuts.

The two are not independent; they meet at the boundaries. The stratum boundaries that the ℓ_1 dimension field detects are precisely the places where the ℓ_2 prediction synchronization should be

relaxed or cut, because across a true stratum boundary the surface itself may be discontinuous and smoothing across it would be wrong. So the dimension field does not merely stabilize the charts; it can *gate* the prediction coupling — smooth within a stratum, sharp across it. Sharp where the geometry is sharp, smooth where it is smooth.

That is the destination this note has been walking toward. We began with a line fitted in a moving window. We found that on real data the window must live on a chart, that the chart's dimension is the most important and least stable number in the method, that stabilizing it means regularizing a field over the neighbor graph, and that the geometry of the data — stratified, compositional, scale-dependent — dictates an ℓ_1 penalty solved by exact combinatorial optimization. The elementary smoother and the deep geometry turn out to be the same story told at two scales.

Appendices: the core ideas, from scratch

Each appendix states one reference's central idea precisely and then re-explains it in plain words. They are self-contained and can be read in any order.

A Bickel & Li (2007): local regression already adapts to a manifold

Core idea. Suppose the covariates lie on (or near) a d-dimensional manifold inside $\mathbb{R}^D$, with $d \ll D$. Bickel and Li show that ordinary local polynomial regression, with a suitably chosen bandwidth, achieves the nonparametric error rate governed by d, the *intrinsic* dimension, rather than D — the estimator never needs to be told the manifold exists. They also give a data-driven bandwidth that uses an estimate of d obtained from the local covariance, making the adaptation automatic.

In plain terms. If your data secretly live on a curved sheet inside a huge space, you do not pay the price of the huge space. The familiar moving-window fit is already smart enough to feel the sheet and run at the speed of the sheet's low dimension — provided the window is sized correctly, which requires knowing the dimension. This is the theorem that makes LPS-on-charts a principled estimator rather than a hopeful heuristic, and it is the first place the local dimension appears as the rate-controlling quantity.

B Cheng & Wu (2013): local linear regression on the tangent plane

Core idea. Cheng and Wu make the manifold construction explicit and analyze it. At each point they estimate the tangent plane by local PCA, reduce the covariates to those d tangent coordinates, and fit a local *linear* model there. They derive the bias and variance, and isolate a term that pure Euclidean local regression does not have: a bias contributed by the manifold's *curvature* and by error in the estimated tangent plane. The estimator is consistent and rate-optimal, and the analysis says exactly when the curvature term matters.

In plain terms. This is the blueprint LPS follows: find the flat approximating plane from the data, then fit a tilt on it. The new subtlety they expose is that the plane is only approximate — a curved surface is not flat, and your estimate of "flat" is itself noisy — so there is an extra, unavoidable error from curvature. It is the precise reason the chart, and hence the chart's *dimension*, has to be handled with care rather than trusted blindly.

C Kpotufe (2011): k-NN regression adapts to the *local* dimension

Core idea. Kpotufe analyzes k-nearest-neighbor regression and proves an error bound at each query point that depends only on the *local* growth of mass in balls around that point — effectively, the local intrinsic dimension there. Where the data are locally low-dimensional the estimator is locally fast; where they are higher-dimensional it is locally slower; and a simple local choice of k nearly achieves the best local rate. Crucially, no single global dimension need be assumed.

In plain terms. Different neighborhoods of the same data set can have different intrinsic dimensions, and the nearest-neighbor method automatically runs at each neighborhood's own speed. This is the theoretical license for letting the dimension vary across the space — LPS's `local.auto` mode — and it makes the local dimension, not some global average, the quantity that has to be estimated well point by point.

D Levina & Bickel (2004): dimension from neighbor-distance growth

Core idea. In a d-dimensional set, the number of sample points within distance r of a query grows like r^d. Treating the neighbors as a Poisson process and writing down the likelihood of the observed nearest-neighbor distances yields a closed-form maximum-likelihood estimate of d at each point: roughly, the inverse average log-ratio of successive neighbor distances. It is a local estimator, computed from a handful of neighbors.

In plain terms. Stand at a point and walk outward; count how fast you accumulate neighbors. In a line they pile up slowly, in a plane faster, in a solid faster still — the rate *is* the dimension. Levina and Bickel turn that count into a formula. It is one of the standard ways LPS could score how well a candidate dimension fits an anchor; like every such estimator it is sharp in principle and noisy from a dozen points, which is exactly the difficulty this note is about.

E Tibshirani et al. (2005): the fused lasso

Core idea. The fused lasso estimates a sequence (or, on a graph, a field) by least squares plus an ℓ_1 penalty on the *differences* of neighboring values, $\lambda \sum_{(i,j)\in E} |\theta_i - \theta_j|$. Because the absolute value does not over-penalize large jumps, the solution is *piecewise constant*: it sets most neighboring differences exactly to zero and concentrates change in a few sharp jumps. On a chain this is total-variation denoising; on a graph it is graph total variation.

In plain terms. It is the "keep the borders" penalty. Telling an estimate "do not change much from your neighbor, and I will charge you the absolute size of any change" produces flat regions separated by crisp edges, because one honest big jump is cheaper than a long smear of little ones. This is exactly the behavior we need for a dimension map, and it is why Section 8 chooses ℓ_1 over ℓ_2.

F Ishikawa (2003): exact graph cuts for convex ordered-label priors

Core idea. Consider labeling every node of a graph with an integer from a linearly ordered set, minimizing per-node costs plus pairwise costs that are *convex* functions of the label difference (total variation is one such). Ishikawa shows this is equivalent to a minimum-cut problem on an auxiliary graph that stacks the labels into layers, and is therefore solved *exactly* in polynomial time. Convexity of the pairwise term is both sufficient and, in a precise sense, necessary for this exactness.

In plain terms. Choosing the best whole-number label at every node of a network, when "disagreement" is measured by a nicely behaved (convex) function, is not the hard combinatorial nightmare it sounds like. There is a clever re-drawing of the problem as cutting a graph into two pieces as cheaply as possible, and that has a fast, exact solution. It is what lets us solve the dimension field optimally instead of approximately.

G Boykov, Veksler & Zabih (2001): α-expansion

Core idea. For labeling energies whose pairwise term is a metric but *not* convex — the Potts penalty $\mathbf{1}\{d_i \neq d_j\}$, which treats every jump equally — exact minimization is NP-hard for three or more labels. α-expansion reduces the problem to a sequence of binary graph cuts: in each round a single label α is allowed to "expand" to take over nodes if doing so lowers the energy. The procedure converges quickly and provably lands within a known constant factor of the global optimum.

In plain terms. When the penalty is the blunt "any change costs one," the exact problem is genuinely hard, but a greedy-but-guaranteed trick works: go through the candidate dimensions one at a time and let each try to claim territory, repeating until nothing improves. You get a near-best map fast. It is the practical fallback when one wants the strict "partition into constant regions" penalty rather than total variation.

H Domokos, Schmidt & Cremers (2018): partially ordered labels

Core idea. Ishikawa's exactness needs a *linear* order on labels. Many labelings are naturally only *partially* ordered — a lattice. Domokos and coauthors extend exact graph-cut optimization with separable convex priors to partially ordered label sets, building the auxiliary graph from the order relation itself.

In plain terms. Sometimes the labels are not a simple low-to-high ladder but a branching hierarchy — like the faces of a simplex, where a face is "below" another if it is part of its boundary. This work shows how to keep the exact, efficient optimization when the labels carry that kind of branching order. It is the principled tool for making the dimension field obey the frontier rule of stratifications (Section 11): dimension may drop toward a seam, not sprout in isolation.

I Haro, Randall & Sapiro: stratification learning

Core idea. Rather than assume a point cloud is one manifold of one dimension, model it as a mixture of pieces of possibly *different* dimensions and densities, and infer, simultaneously, a soft assignment of points to pieces, each piece's intrinsic dimension, and its density. A Poisson / mixture likelihood built on local neighbor-distance statistics (in the Levina–Bickel spirit) drives the inference.

In plain terms. It is manifold learning that admits the world is messier than one smooth sheet: data can be a one-dimensional thread, a two-dimensional sheet and a denser blob all at once, and the method sorts points into those pieces while measuring each piece's dimension. It is the statistical ancestor of the dimension-field idea, naming the goal — recover the strata and their dimensions — that this note pursues with a graph-regularized, edge-preserving estimator.

J Bendich, Wang & Mukherjee: local homology and stratification

Core idea. The *local homology* at a point — the homology of a small neighborhood relative to its boundary, or of the neighborhood with the point removed — is a topological fingerprint that distinguishes a smooth d-dimensional spot from a singular one (an edge, a crossing, a cone tip). Bendich, Wang and Mukherjee develop how to estimate and *transfer* local homology across nearby sample points, turning it into a tool for detecting strata and their dimensions from finite, noisy samples.

In plain terms. Poke a tiny hole at a point and look at the shape of what surrounds it: around an ordinary point of a sheet it looks like a disk with its center missing; at an edge or a junction it looks different. That "shape of the immediate surroundings" detects where the dimension changes and what kind of singularity sits there. It is a topological, rather than spectral, way to read the local dimension and to find the seams between strata.

K Multi-scale singularity and dimension detection

Core idea. Because intrinsic dimension and "singularity vs. smooth" are scale-dependent (Section 9), a robust verdict comes from watching how a local geometric or topological measure *changes across scales* rather than fixing one radius. Recent methods (e.g. "Topological Singularity Detection at Multiple Scales"; the HADES local-measure-comparison detector) flag a point as singular, or assign its dimension, by comparing local structure across a range of neighborhood sizes and looking for the scales at which it is stable or breaks.

In plain terms. Do not judge a point's dimension from a single zoom level; sweep the zoom and watch. A genuine feature persists across a band of scales; a fluke appears at one scale and vanishes at another. This is the rigorous version of the "trust a persistent dimension" rule, and it is how to make the per-anchor cost $c_i(d)$ robust to the scale problem.

References

[1] P. J. Bickel and B. Li, Local polynomial regression on unknown manifolds, in *Complex Datasets and Inverse Problems*, IMS Lecture Notes 54 (2007).

[2] M.-Y. Cheng and H.-T. Wu, Local linear regression on manifolds and its geometric interpretation, *J. Amer. Statist. Assoc.* 108 (2013) 1421–1434.

[3] S. Kpotufe, k-NN regression adapts to local intrinsic dimension, *NeurIPS* (2011).

[4] E. Levina and P. J. Bickel, Maximum likelihood estimation of intrinsic dimension, *NeurIPS* (2004).

[5] R. Tibshirani, M. Saunders, S. Rosset, J. Zhu and K. Knight, Sparsity and smoothness via the fused lasso, *J. R. Statist. Soc. B* 67 (2005) 91–108.

[6] H. Ishikawa, Exact optimization for Markov random fields with convex priors, *IEEE Trans. Pattern Anal. Mach. Intell.* 25 (2003) 1333–1336.

[7] Y. Boykov, O. Veksler and R. Zabih, Fast approximate energy minimization via graph cuts, *IEEE Trans. Pattern Anal. Mach. Intell.* 23 (2001) 1222–1239.

[8] C. Domokos, F. R. Schmidt and D. Cremers, MRF optimization with separable convex prior on partially ordered labels, *ECCV* (2018).

[9] G. Haro, G. Randall and G. Sapiro, Stratification learning: detecting mixed density and dimensionality in high dimensional point clouds, *NeurIPS* (2006); expanded in *Int. J. Comput. Vis.* (2008).

[10] P. Bendich, B. Wang and S. Mukherjee, Local homology transfer and stratification learning, *ACM–SIAM SODA* (2012); see also *Approximating local homology from samples*, SODA (2014).

[11] *Topological Singularity Detection at Multiple Scales*, arXiv:2210.00069; *HADES: Fast Singularity Detection with Local Measure Comparison*, arXiv:2311.04171.

[12] Classical local polynomial regression: W. S. Cleveland (LOWESS, 1979); J. Fan and I. Gijbels, *Local Polynomial Modelling and Its Applications* (1996); C. Loader, *Local Regression and Likelihood* (1999).